CASE STUDY — SUPPORTVECTORS AI AGENTS BOOTCAMP CAPSTONE

The Disclosure Verifier

An agentic system that checks whether a company's own words match its own numbers — extracting quantitative claims from SEC filing prose and reconciling each one against the structured data the company itself reported, **with a citation back to the exact filing.**

Status Phases 0–5 complete, Phase 6–7 pending **Tests** 107 passing, CI green

Repo github.com/asim-aa/disclosure-verifier

"Revenue for fiscal year 2026 was $215.9 billion, up 65% from a year ago."

extracted → `{metric: Revenue, value: 215,900,000,000, type: absolute}`

extracted → `{metric: Revenue, value: 65.0, type: growth_pct}`

reconciled → `CONSISTENT` EDGAR reports $215,938,000,000 — 0.02% off

reconciled → `INCONSISTENT` EDGAR implies 46.09% growth, not 65%

01 DSPy optimization, measured against a baseline

The capstone's Pillar 2 requirement: at least one prompt signature optimized with DSPy, scored against a hand-written baseline — not just used, *proven*. Ground truth is 78 examples / 161 claims / 9 true negatives, hand-labeled from real MSFT and NVDA filings before any DSPy code was written, to keep the measurement honest.

EXTRACTOR	PRECISION	RECALL	F1
Hand-written baseline prompt	0.711	0.750	0.730
DSPy zero-shot (ChainOfThought, no demos)	0.763	0.806	0.784
DSPy optimized (BootstrapFewShot)	**0.763**	**0.806**	**0.784**

Held-out test set, 16 examples never seen during optimization. Delta vs. baseline:
+0.054 F1

The honest framing matters more than the win — including a result that didn't go the way you'd expect. Switching the signature from `Predict` to `ChainOfThought` — reasoning before committing to structured output — took zero-shot DSPy from underperforming the baseline (an earlier run measured 0.676 F1 on the bare `Predict` signature) to clearly beating it (0.784), before any optimization ran. But `BootstrapFewShot` optimization on top of that produced *exactly identical* numbers: same precision, same recall, the same 29/9/7 true-positive/false-positive/false-negative split. Verified this wasn't an evaluation bug — the compiled program is a genuinely separate instance, and the optimizer log confirms 4 real demonstrations were bootstrapped. The reasoning step captured essentially all of the available signal; the few-shot demos added nothing further. Reasoning mattered more than optimization for this task — a more interesting and more honest finding than a clean "optimization wins" story would have been.

02 A real run, live

Not a cached demo — this is the actual coordinator, run against NVDA's FY2026 10-K, live EDGAR data and a live LLM call for every extraction.

NVDA · FY2026 10-K · MD&A		**13 claims verified**
CONSISTENT	Revenue EDGAR: $215,938,000,000 — 0.02% off	$215.9B
INCONSISTENT	Revenue growth EDGAR implies 46.09% growth, not the stated figure	65.0%
CONSISTENT	Income tax expense EDGAR: $21,383,000,000 — 0.08% off	$21.4B
UNVERIFIABLE	Data Center revenue growth segment-level — no top-level XBRL tag exists; declined rather than guessed	68.0%

13	22	1.39s	0
claims checked	tool calls	wall-clock	crashes

03 What actually broke

The interesting engineering here wasn't the happy path — it was what real SEC data did to the happy path. Each of these was caught by testing against live data, not by trusting the design on paper.

---

01 A metric-matching shortcut that would verify the wrong number

Resolving a claim's free-text metric ("Azure revenue") to an exact XBRL concept originally tolerated wording variance via substring matching. But `"revenue"` is a substring of `"Azure and other cloud services revenue"` — so a segment's revenue would have been silently checked against *total company* revenue and returned a confidently wrong "consistent" verdict. Fixed to exact-match plus explicit aliases only.

`agents/resolver.py`

---

02 XBRL doesn't tag percentages as percentages

Rate concepts like effective tax rate are reported with `unit="pure"` — a decimal fraction (`0.19` , not `19`). A claim stating "19%" would have failed to match the fact at all, and once that was fixed, would have compared `19.0` against `0.19` and read as wildly wrong even when correct.

`agents/verification_agent.py`

---

03 Companies retag their own concepts mid-history

NVDA reported revenue under `RevenueFromContractWithCustomerExcludingAssessedTax` through FY2022, then switched to plain `Revenues` . The old tag never disappears from the company's concept list — it just stops receiving new data. Picking "the first candidate that exists at all" silently locked onto a tag with no data newer than 2022. Fixed to pick by data recency instead.

`agents/resolver.py`

04

A newer filing can corrupt an older one's claims

A claim extracted from a 10-K's MD&A describes *that 10-K's* fiscal year — but a newer 10-Q, almost always already filed by the time verification runs, has a more recent quarterly `period_end`. Without a cutoff, that quarterly figure would get picked as "current," comparing the wrong two numbers entirely. Fixed with an `as_of` anchor tied to the claim's own source filing.

`agents/resolver.py`

05

A bug in the measurement itself, caught before it could lie

An early precision/recall scorer would have counted a generic prediction of `"Revenue"` as matching gold label `"Microsoft Cloud revenue"` — a bug in the *grader*, which is worse than a bug in the thing being graded, because it makes bad results look good. Its own unit test caught it before the real comparison ever ran.

`eval/metrics.py`

06

Reasoning silently corrupted its own numeric output

Switching the extraction signature from `dspy.Predict` to `dspy.ChainOfThought` made the model write dollar amounts in shorthand inside its reasoning — *"$215.9 billion"* — then just echo that literal `215.9` into the structured `value` field instead of expanding it to `215900000000`. Consistent, every run. Plain `Predict` never had this failure mode, because it never generates that intervening shorthand text to anchor on. Fixed by instructing the signature to write the fully-expanded number in the reasoning itself, so the structured step has nothing left to get wrong.

`eval/dspy_extractor.py`

> **OPEN, NOT HIDDEN**
>
> The verification agent still can't parse a claim's *stated* period text — it distinguishes "the source filing's own period" from "the next most recent one," but not "this sentence's FY2025 figure" from "this sentence's FY2026 figure" when both appear together. A claim about an explicitly non-current period can still resolve against the wrong one. Documented as follow-up in `agents/resolver.py`, not silently left broken.

Written up as part of an ongoing capstone build.

View the repository →